

DGTokenVendor Smart Contract: Comprehensive Knowledge Base

This document provides a complete and detailed overview of the DGTokenVendor smart contract, combining all aspects of its functionality, configuration, and rules.

DGTokenVendor Smart Contract: Overview

1. Primary Purpose

DG Token Vendor is a decentralized application that provides a systematic framework for fostering community engagement and rewarding user participation in Web3. It addresses the challenge of low retention and intimidating onboarding by transforming community involvement into a structured, gamified experience.

At its core, DG Token Vendor is an exclusive, NFT-gated token exchange. To interact with its primary functions—buying and selling of a “base” token for a “swap” token. A user must first hold a valid NFT from a whitelisted collection. This mechanism creates a clear entry point and pathway for progression.

Once inside the ecosystem, the application incentivizes long-term participation through a multi-tiered progression system. A user begins at the ‘Pleb’ stage and can strategically advance to ‘Hustler’ and ‘OG’ status by accumulating points and ‘fuel’ through defined actions, such as making qualifying token purchases. Each stage unlocks progressively better rewards and privileges, such as more favorable transaction terms or higher limits.

By replacing arbitrary reward methods with a transparent, rule-based journey, DG Token Vendor provides a predictable and scalable tool for building and sustaining a loyal, active, and deeply engaged community.

2. Core Concepts

The contract is built around several key concepts that work together:

- **Token Swapping:** The fundamental feature is the ability to buy `swapToken` with `baseToken` and sell `swapToken` for `baseToken`. The exchange rate is configurable by the contract owner.
- **NFT-Gated Access:** To interact with the core functions like buying or selling tokens, a user **must** hold a valid key from a whitelisted NFT collection (utilizing the `PublicLock` interface). This makes the vendor exclusive to members of specific communities.
- **User Stages:** Users progress through a tier system with three levels: `PLEB`, `HUSTLER`, and `OG`. Each stage has different parameters, offering better

terms or higher limits as a user advances. Progression is not automatic and requires users to meet specific criteria.

- **Points System:** Users earn “points” by performing certain actions, primarily by making qualifying purchases of the swap token. These points are a key requirement for upgrading to a higher stage.
- **Fuel System:** “Fuel” is another resource users can accumulate. It is primarily gained through a “light up” mechanism, which involves burning a small number of tokens. Fuel is another requirement for stage upgrades and can also be used to temporarily increase a user’s daily sell limit.
- **Fees:** The contract applies fees to both buy and sell transactions. These fees are collected in their respective tokens (**baseToken** for buys, **swapToken** for sells) and can be withdrawn by an authorized address.

3. Key Roles & Authorizations

The contract defines several distinct roles with specific permissions:

- **User (NFT Holder):** A regular user who holds a valid NFT key. They can access the primary functions: **buyTokens**, **sellTokens**, **lightUp**, and **upgradeStage**.
- **Owner:** The ultimate administrator of the contract. The owner has the highest level of authority and can manage all critical settings, including fee rates, exchange rates, stage configurations, and whitelisted NFT collections. The owner can also pause the contract and appoint a new Steward Council.
- **Developer (Dev Address):** An authorized address set by the owner. This address is designated to receive the fees collected by the contract when **withdrawFees** or **withdrawETH** is called. The Dev can also update their own address.
- **Steward Council:** A secondary administrative address (intended to be a multisig wallet) that shares some administrative powers with the owner. It can pause/unpause the contract, providing a layer of security and decentralized control.
- **Anyone:** Some functions, primarily **view** functions that read data (e.g., **getExchangeRate**, **getStageConfig**), can be called by any address without requiring special permissions.

DGTokenVendor Smart Contract: State & Configuration

This document details the state variables, constants, and data structures (structs) that form the data model of the **DGTokenVendor** contract.

1. Core Constants

These are fixed, unchangeable values that define global limits and parameters for the contract's logic.

- **MAX_WHITELISTED_COLLECTIONS: 10**
 - The maximum number of NFT collections that can be whitelisted for access.
- **BASIS_POINTS: 10000**
 - The denominator for calculating percentage-based fees (e.g., a fee of 100 BPS is $100/10000 = 1\%$).
- **MAX_DAILY_MULTIPLIER: 100**
 - The maximum multiplier that can be applied to the `qualifyingBuyThreshold` to determine a user's daily sell limit.
- **MAX_FUEL_LIMIT: 100**
 - The absolute maximum amount of “fuel” a user can accumulate.
- **MAX_FUEL_RATE: 5**
 - The maximum rate at which fuel can be earned per `lightUp` action.
- **MAX_POINTS_AWARDED: 5**
 - The maximum number of points that can be awarded for a qualifying buy.
- **MAX_SELL_BPS_LIMIT: 7000**
 - The maximum percentage (in basis points, so 70%) of the contract's balance that a user in the highest stage can sell in a single transaction.
- **BURN_ADDRESS: 0x2Ef7DeC913e4127Fd0f94B32eeAd23ee63143598**
 - The designated address where tokens are sent to be burned during the `lightUp` process.

2. Data Structures (Structs)

Structs are custom data types that group related variables together, organizing the contract's data.

StageConstants

Stores global parameters related to the stage and timing mechanics.

- **maxSellCooldown (uint256):** The cooldown period (in seconds) that an OG stage user must wait after making a maximum-sized sale before they can make another one.
- **dailyWindow (uint256):** The duration (in seconds) of the rolling window for calculating a user's daily sell limit.
- **minBuyAmount (uint256):** The minimum amount of `baseToken` a user must spend in a single `buyTokens` transaction.
- **minSellAmount (uint256):** The minimum amount of `swapToken` a user must sell in a single `sellTokens` transaction.

FeeConfig

Stores all parameters related to transaction fees and administrative cooldowns.

- **maxFeeBps** (uint256): The maximum fee (in basis points) that can be set for buys or sells.
- **minFeeBps** (uint256): The minimum fee (in basis points) that can be set.
- **buyFeeBps** (uint256): The current fee percentage for **buyTokens** transactions.
- **sellFeeBps** (uint256): The current fee percentage for **sellTokens** transactions.
- **rateChangeCooldown** (uint256): The minimum time (in seconds) that must pass before the **exchangeRate** can be changed again.
- **appChangeCooldown** (uint256): The minimum time (in seconds) that must pass before general app settings (like fees or the dev address) can be changed again.

TokenConfig

Stores the contract addresses and configuration for the tokens involved in the exchange.

- **baseToken** (IERC20): The contract address of the token used to purchase the **swapToken** (e.g., USDC).
- **swapToken** (IERC20): The contract address of the token being sold by the vendor (e.g., DG Token).
- **exchangeRate** (uint256): The rate at which **baseToken** is converted to **swapToken**. For example, a rate of 100 means 1 **baseToken** buys 100 **swapToken**.

SystemState

Stores system-wide state variables that track fees and important timestamps.

- **baseTokenFees** (uint256): The total amount of **baseToken** fees collected and waiting for withdrawal.
- **swapTokenFees** (uint256): The total amount of **swapToken** fees collected and waiting for withdrawal.
- **lastRateChangeTimestamp** (uint256): The timestamp of the last time the **exchangeRate** was modified.
- **lastFeeChangeTimestamp** (uint256): The timestamp of the last time the **buyFeeBps** or **sellFeeBps** were modified.
- **devAddress** (address): The designated address for withdrawing collected fees.
- **stewardCouncil** (address): The address of the multisig or secondary admin.
- **lastDevAddressChangeTimestamp** (uint256): The timestamp of the last time the **devAddress** was changed.

UserState

Stores all the data specific to an individual user.

- **stage** (UserStage): The user's current progression stage (PLEB, HUSTLER, or OG).
- **points** (uint256): The number of points the user has accumulated towards the next stage.
- **fuel** (uint256): The user's current fuel level.
- **lastStage3MaxSale** (uint256): The timestamp of the last time the user, as an OG, performed a maximum-sized sale.
- **dailySoldAmount** (uint256): The total amount of **swapToken** the user has sold within the current **dailyWindow**.
- **dailyWindowStart** (uint256): The timestamp marking the beginning of the user's current daily tracking period.

StageConfig

Stores the specific parameters and thresholds for each **UserStage**.

- **burnAmount** (uint256): The amount of **baseToken** a user at this stage must burn to use the **lightUp** feature.
- **upgradePointsThreshold** (uint256): The number of points required to upgrade *to* this stage.
- **upgradeFuelThreshold** (uint256): The amount of fuel required to upgrade *to* this stage.
- **fuelRate** (uint256): The amount of fuel gained per **lightUp** action at this stage.
- **pointsAwarded** (uint256): The number of points awarded for a qualifying purchase at this stage.
- **qualifyingBuyThreshold** (uint256): The minimum **buyTokens** amount required to earn points at this stage.
- **maxSellBps** (uint256): The maximum percentage of the contract's **baseToken** balance that a user at this stage can receive from a single sale.
- **dailyLimitMultiplier** (uint256): The multiplier used with **qualifyingBuyThreshold** to calculate the daily sell limit for a user at this stage.

3. State Variables

These are the top-level variables where the contract's state is stored.

- **stageConstants** (StageConstants): An instance of the **StageConstants** struct, holding the global timing and amount settings.
- **feeConfig** (FeeConfig): An instance of the **FeeConfig** struct.
- **tokenConfig** (TokenConfig): An instance of the **TokenConfig** struct.
- **systemState** (SystemState): An instance of the **SystemState** struct.
- **whitelistedCollections** (address[]): A dynamic array storing the addresses of the whitelisted NFT collections.

- **userStates** (mapping(address => UserState)): A mapping that links a user's wallet address to their individual **UserState** struct.
- **stageConfig** (mapping(UserStage => StageConfig)): A mapping that links each **UserStage** enum to its specific **StageConfig** struct.

4. Default Initial Configuration

These are the default values set in the `_initialize` function upon contract deployment.

FeeConfig Defaults

- **maxFeeBps**: **1000** (10%)
- **minFeeBps**: **10** (0.1%)
- **buyFeeBps**: **100** (1%)
- **sellFeeBps**: **200** (2%)
- **rateChangeCooldown**: **90 days**
- **appChangeCooldown**: **120 days**

StageConstants Defaults

- **maxSellCooldown**: **45 days**
- **dailyWindow**: **24 hours**
- **minBuyAmount**: **1,000 tokens** (1000e18)
- **minSellAmount**: **5,000 tokens** (5000e18)

StageConfig Defaults

PLEB (Stage 0)

- **burnAmount**: **10 tokens** (10e18)
- **upgradePointsThreshold**: **0** (Not applicable)
- **upgradeFuelThreshold**: **0** (Not applicable)
- **fuelRate**: **1**
- **pointsAwarded**: **1**
- **qualifyingBuyThreshold**: **1,000 tokens** (1000e18)
- **maxSellBps**: **5000** (50%)
- **dailyLimitMultiplier**: **100**

HUSTLER (Stage 1)

- **burnAmount**: **100 tokens** (100e18)
- **upgradePointsThreshold**: **50**
- **upgradeFuelThreshold**: **10**
- **fuelRate**: **2**
- **pointsAwarded**: **2**
- **qualifyingBuyThreshold**: **5,000 tokens** (5000e18)

- **maxSellBps: 6000 (60%)**
- **dailyLimitMultiplier: 100**

OG (Stage 2)

- **burnAmount: 500 tokens (500e18)**
 - **upgradePointsThreshold: 500**
 - **upgradeFuelThreshold: 100**
 - **fuelRate: 5**
 - **pointsAwarded: 5**
 - **qualifyingBuyThreshold: 20,000 tokens (20000e18)**
 - **maxSellBps: 7000 (70%)**
 - **dailyLimitMultiplier: 100**
-

DGTokenVendor Smart Contract: Access Control

This document describes the access control mechanisms and roles within the **DGTokenVendor** smart contract. Access control is primarily managed through a combination of ownership checks and custom modifiers that restrict certain functions to authorized addresses.

1. Core Roles

The contract defines several key roles, each with different permissions:

- **User (NFT Holder):** The standard user who must hold a valid NFT from a whitelisted collection to access the main features.
- **Owner:** The primary administrator with full control over the contract's settings.
- **Developer (Dev):** A designated address responsible for fee collection and updating its own address.
- **Steward Council:** A secondary administrative body (like a multisig) with the ability to pause and unpause the contract.
- **Authorized:** A role that includes both the **Owner** and the **Developer**, used for functions they are both permitted to call (e.g., withdrawing fees).
- **Admin:** A role that includes both the **Owner** and the **Steward Council**, used for functions they are both permitted to call (e.g., pausing the contract).

2. Access Control Modifiers

Modifiers are reusable pieces of code that check for certain conditions before allowing a function to execute. They are the primary tool for enforcing permissions.

`onlyNFTHolder()`

This modifier ensures that the function caller holds a valid NFT key from one of the collections in the `whitelistedCollections` array.

- **Check:** It calls the internal `hasValidKey(msg.sender)` function.
- **Behavior:** If the user does not have a valid key, the transaction is reverted with a `NoValidKeyForUserFound()` error.
- **Used In:**
 - `buyTokens(uint256 amount)`
 - `sellTokens(uint256 amount)`
 - `lightUp()`
 - `upgradeStage()`

`onlyAuthorized()`

This modifier restricts a function to be callable only by the contract `owner` or the `systemState.devAddress`.

- **Check:** It verifies if `msg.sender` is either the `owner()` or the `devAddress`.
- **Behavior:** If the caller is not one of these two addresses, the transaction is reverted with an `UnauthorizedCaller()` error.
- **Used In:**
 - `withdrawFees()`
 - `withdrawETH()`
 - `initializeWhitelistedCollections(address[] calldata collections)`

`onlyDev()`

This modifier restricts a function to be callable only by the `systemState.devAddress`.

- **Check:** It verifies if `msg.sender` is the `devAddress`.
- **Behavior:** If the caller is not the developer, the transaction is reverted with an `UnauthorizedCaller()` error.
- **Used In:**
 - `setDevAddress(address newDevAddress)`

`onlyAdmin()`

This modifier restricts a function to be callable only by the contract `owner` or the `systemState.stewardCouncil` address.

- **Check:** It verifies if `msg.sender` is either the `owner()` or the `stewardCouncil`.
- **Behavior:** If the caller is not one of these two addresses, the transaction is reverted with an `UnauthorizedCaller()` error.
- **Used In:**
 - `pause()`

- `unpause()`

`onlyOwner()`

This is a standard modifier from OpenZeppelin’s `Ownable` contract. It restricts a function to be callable only by the contract’s owner.

- **Check:** It verifies if `msg.sender` is the `owner()`.
- **Behavior:** If the caller is not the owner, the transaction is reverted.
- **Used In:**
 - `setExchangeRate(uint256 newRate)`
 - `setFeeRates(uint256 newBuyFeeBPS, uint256 newSellFeeBPS)`
 - `setStewardCouncilAddress(address _newCouncilAddress)`
 - `setStageConfig(UserStage _stage, StageConfig calldata _config)`
 - `setCooldownConfig(uint256 _rateChangeCooldown, uint256 _appChangeCooldown)`

3. Whitelisted NFT Collections

To use the core functions of the `DGTokenVendor`, users must hold a valid NFT from one of the following whitelisted collections. Each collection has different purposes and durations.

- **P2E INFERNO IGNITION**
 - **Description:** Awarded for successfully completing the Infernal Sparks Bootcamp — A 4-week onboarding journey into the P2E Inferno ecosystem.
 - **Duration:** 30 Days
 - **How to Obtain:** This NFT cannot be purchased. It is awarded to participants who complete all milestones in the Infernal Sparks Bootcamp.
- **DG Nation**
 - **Description:** The beating heart and flickering flame that keeps the engine of the `DGToken Vendor` running.
 - **Duration:** 30 Days
 - **How to Obtain:** `DG Nation NFT` is a monthly subscription that can be purchased for 10000 `DGT`.
- **DGToken Vendor Sponsor**
 - **Description:** A soulbound (non-transferable) NFT for sponsors who want a front-row seat at the foundation of the `DGToken Vendor`.
 - **Duration:** 60 Days
 - **How to Obtain:** Can be purchased for 20000 `UP`.
- **DGToken Vendor Supporter**
 - **Description:** Show your support for our Digital Game ecosystem with this NFT collection. Holders get exclusive benefits and early access to features.

- **Duration:** 180 Days
- **How to Obtain:** Can be purchased for 500 USDC.
- **DG Nation Tourist**
 - **Description:** An access pass for short-term visitors of DG Nation looking to explore and experience the frontiers.
 - **Duration:** 1 Day
 - **How to Obtain:** Can be purchased for 0.005 ETH.
- **DGToken CEx**
 - **Description:** Designed for long-term players who want to participate as Point Of Sale vendors exchanging DGTokens for fiat.
 - **Duration:** 365 Days
 - **How to Obtain:** Can be purchased for 100000 UP.

4. Other Important Modifiers

These modifiers are not for access control but are critical for contract safety and state management.

- **nonReentrant():** From OpenZeppelin’s **ReentrancyGuard**, this modifier protects a function from re-entrancy attacks, where a malicious contract calls back into the function before the first call is complete.
- **whenNotPaused():** From OpenZeppelin’s **Pausable**, this modifier ensures that a function can only be executed when the contract is not in a paused state.

DGTokenVendor Smart Contract: User Functions

This document explains the primary functions that are intended for direct use by end-users (NFT holders). Access to all these functions is protected by the **onlyNFTHolder** modifier, meaning the caller must have a valid key from a whitelisted NFT collection.

1. **buyTokens(uint256 amount)**

This function allows a user to purchase **swapToken** by spending their **baseToken**.

- **Purpose:** To exchange **baseToken** for **swapToken**.
- **Parameter:**
 - **amount** (uint256): The amount of **baseToken** the user wishes to spend.
- **Access Control:** **onlyNFTHolder**, **whenNotPaused**

Process Flow:

1. **Minimum Amount Check:** It first checks if the `amount` is greater than or equal to `stageConstants.minBuyAmount`. If not, it reverts with `MinimumAmountNotMet()`.
2. **Balance Check:** It verifies that the user has enough `baseToken` to cover the `amount`. If not, it reverts with `InsufficientBalance()`.
3. **Fee Calculation:** A `buyFeeBps` percentage is calculated from the `amount` and set aside as a fee.
4. **Token Calculation:** The remaining amount (after the fee) is multiplied by the current `tokenConfig.exchangeRate` to determine how much `swapToken` the user will receive.
5. **Points Award:** If the `amount` spent is greater than or equal to the `qualifyingBuyThreshold` for the user's current stage, the user is awarded `pointsAwarded` for their stage. This is how users accumulate points for stage upgrades.
6. **Token Transfers:**
 - The full amount of `baseToken` is transferred from the user to the contract.
 - The calculated `swapToken` amount is transferred from the contract to the user.
7. **Event:** Emits a `TokensPurchased` event with the details of the transaction.

2. `sellTokens(uint256 amount)`

This function allows a user to sell their `swapToken` to receive `baseToken` in return.

- **Purpose:** To exchange `swapToken` for `baseToken`.
- **Parameter:**
 - `amount (uint256)`: The amount of `swapToken` the user wishes to sell.
- **Access Control:** `onlyNFTHolder`, `whenNotPaused`

Process Flow:

1. **Minimum Amount Check:** It checks if the `amount` is at least `stageConstants.minSellAmount`. If not, it reverts.
2. **Fee Calculation:** A `sellFeeBps` percentage is calculated from the `amount` and set aside as a fee.
3. **Token Calculation:** The remaining `swapToken` amount is divided by the `exchangeRate` to determine how much `baseToken` the user will receive.
4. **Sell Limit Checks:** This is the most complex part of the function:
 - **Stage Sell Limit:** It calculates the maximum single transaction sell amount (`maxTxSell`) based on the contract's `baseToken` balance and the user's stage-specific `maxSellBps`. If the user tries to receive more `baseToken` than this limit, the transaction reverts with `StageSellLimitExceeded()`.

- **Daily Sell Limit:** It checks and updates the user's `dailySoldAmount` against a calculated `dailyLimit`. This daily limit is determined by the user's `qualifyingBuyThreshold`, `dailyLimitMultiplier`, and their current `fuel` level. Selling consumes any available `fuel`. If the daily limit is exceeded, it reverts with `DailySellLimitExceeded()`.
 - **OG Cooldown:** If the user is at the OG stage and performs a `maxTxSell`, a cooldown (`stageConstants.maxSellCooldown`) is initiated, preventing another max-sized sale until the cooldown expires.
5. **State Updates:**
 - The user's `dailySoldAmount` is increased.
 - The user's `fuel` is reset to 0.
 - The collected `swapTokenFees` in the contract are increased.
 6. **Token Transfers:**
 - The amount of `swapToken` is transferred from the user to the contract.
 - The calculated `baseToken` amount is transferred from the contract to the user.
 7. **Event:** Emits a `TokensSold` event with the transaction details.

3. `lightUp()`

This function allows a user to burn a small amount of `baseToken` to increase their `fuel` level.

- **Purpose:** To gain `fuel`, which is a resource needed for stage upgrades and for temporarily increasing daily sell limits.
- **Parameters:** None.
- **Access Control:** `onlyNFTHolder`, `whenNotPaused`

Process Flow:

1. **Token Transfer:** The contract transfers a `burnAmount` of `baseToken` (determined by the user's stage) from the user to the `BURN_ADDRESS`.
2. **Fuel Update:** The user's `fuel` is increased by the `fuelRate` corresponding to their stage. The new fuel level is capped at `MAX_FUEL_LIMIT`.
3. **Event:** Emits a `Lit` event, logging the user, the amount burned, and their new fuel level.

4. `upgradeStage()`

This function allows a user to advance to the next `UserStage` if they meet the requirements.

- **Purpose:** To move from PLEB -> HUSTLER or HUSTLER -> OG.
- **Parameters:** None.
- **Access Control:** `onlyNFTHolder`, `whenNotPaused`

Process Flow:

1. **Max Stage Check:** It first checks if the user is already at the highest stage (0G). If so, it reverts with `MaxStageReached()`.
 2. **Requirement Checks:** It verifies that the user has met the upgrade criteria for the *next* stage:
 - `user.points` must be $\geq$ `stageConfig[nextStage].upgradePointsThreshold`.
 - `user.fuel` must be $\geq$ `stageConfig[nextStage].upgradeFuelThreshold`.
 - If either check fails, it reverts with `InsufficientPointsForUpgrade()` or `InsufficientFuelForUpgrade()`.
 3. **State Update:**
 - The user's `stage` is incremented to the next level.
 - The user's `points` and `fuel` are reset to 0.
 4. **Event:** Emits a `StageUpgraded` event, announcing the user's new stage.
-

DGTokenVendor Smart Contract: Admin Functions

This document details the administrative functions of the `DGTokenVendor` contract. These functions are restricted to authorized roles (`Owner`, `Dev`, `Steward Council`, `Admin`) and are used to manage the contract's parameters, security, and funds.

1. Ownership and High-Level Control

These functions are typically restricted to the `onlyOwner` or `onlyAdmin` modifiers.

`pause()` & `unpause()`

- **Purpose:** To halt or resume the core functions of the contract in an emergency.
- **Access:** `onlyAdmin` (callable by `Owner` and `Steward Council`).
- **Action:** Sets the contract's `paused` state to `true` or `false`. When paused, modifiers like `whenNotPaused` will cause functions like `buyTokens` and `sellTokens` to revert.

`setStewardCouncilAddress(address _newCouncilAddress)`

- **Purpose:** To update the address of the `Steward Council`.
- **Access:** `onlyOwner`.
- **Action:** Changes the `systemState.stewardCouncil` address. Reverts if the new address is the zero address.
- **Event:** `StewardCouncilAddressUpdated`

2. Financial and Rate Management

These functions control the economic parameters of the token vendor.

setExchangeRate(uint256 newRate)

- **Purpose:** To set the exchange rate between the `baseToken` and `swapToken`.
- **Access:** `onlyOwner`.
- **Action:** Updates `tokenConfig.exchangeRate`. It includes a cooldown (`feeConfig.rateChangeCooldown`) to prevent rapid changes and validates that the `newRate` is not zero and not excessively high.
- **Event:** `ExchangeRateUpdated`

setFeeRates(uint256 newBuyFeeBPS, uint256 newSellFeeBPS)

- **Purpose:** To update the percentage fees for buying and selling tokens.
- **Access:** `onlyOwner`.
- **Action:** Updates `feeConfig.buyFeeBps` and `feeConfig.sellFeeBps`. It includes a cooldown (`feeConfig.appChangeCooldown`) and ensures the new fees are within the allowed `minFeeBps` and `maxFeeBps` limits.
- **Event:** `FeeRatesUpdated`

withdrawFees()

- **Purpose:** To withdraw the accumulated `baseToken` and `swapToken` fees from the contract.
- **Access:** `onlyAuthorized` (callable by `Owner` and `Dev`).
- **Action:** Transfers the entire balance of `systemState.baseTokenFees` and `systemState.swapTokenFees` to the `systemState.devAddress` and resets the fee counters to zero.
- **Event:** `FeesWithdrawn`

withdrawETH()

- **Purpose:** To withdraw any Ether (ETH) that may have been accidentally sent to the contract.
- **Access:** `onlyAuthorized` (callable by `Owner` and `Dev`).
- **Action:** Transfers the entire ETH balance of the contract to the `systemState.devAddress`.
- **Event:** `ETHWithdrawn`

3. Configuration and System Settings

These functions manage the operational parameters of the contract's features.

setDevAddress(address newDevAddress)

- **Purpose:** To change the address that receives withdrawn fees.
- **Access:** onlyDev.
- **Action:** Updates `systemState.devAddress`. This function has its own cooldown (`feeConfig.appChangeCooldown`) and can only be called by the current developer, allowing them to transfer their role.
- **Event:** DevAddressUpdated

initializeWhitelistedCollections(address[] calldata collections)

- **Purpose:** To set the initial list of NFT collections that grant access to the vendor.
- **Access:** onlyAuthorized (callable by Owner and Dev).
- **Action:** Populates the `whitelistedCollections` array. This function can only be called once and will revert if the list is already populated, ensuring the initial setup is immutable.
- **Event:** WhitelistedCollectionAdded for each collection.

setStageConfig(UserStage _stage, StageConfig calldata _config)

- **Purpose:** To update the specific parameters for any of the user stages (PLEB, HUSTLER, OG).
- **Access:** onlyOwner.
- **Action:** Allows the owner to fine-tune the entire configuration for a stage, including burn amounts, upgrade thresholds, fuel rates, points awarded, and sell limits. It performs numerous validation checks to ensure the new parameters are within safe and logical bounds.
- **Event:** StageConfigUpdated

setCooldownConfig(uint256 _rateChangeCooldown, uint256 _appChangeCooldown)

- **Purpose:** To update the cooldown periods for administrative actions.
- **Access:** onlyOwner.
- **Action:** Modifies `feeConfig.rateChangeCooldown` and `feeConfig.appChangeCooldown`. The new values must be within a predefined range (e.g., between 14 and 180 days) to prevent them from being set to trivial or excessively long durations.
- **Event:** FeeConfigUpdated

DGTokenVendor Smart Contract: View Functions

This document describes the `view` functions in the `DGTokenVendor` contract. These functions are read-only, meaning they do not modify the contract's state and do not cost any gas to call (when accessed externally). They are used to

retrieve data about the contract’s configuration, system state, and individual user data.

1. User-Specific View Functions

These functions provide information about a specific user.

`getUserState(address user)`

- **Purpose:** To retrieve the entire `UserState` struct for a given user address.
- **Parameter:**
 - `user` (address): The address of the user to query.
- **Returns:** A `UserState` struct containing the user’s stage, points, fuel, and other personal metrics.

`hasValidKey(address user)`

- **Purpose:** To check if a user holds a valid NFT from any of the whitelisted collections.
- **Parameter:**
 - `user` (address): The address of the user to check.
- **Returns:** `true` if the user has a valid key, `false` otherwise.

`getFirstValidCollection(address user)`

- **Purpose:** To find and return the address of the first whitelisted NFT collection in which the user holds a valid key.
- **Parameter:**
 - `user` (address): The address of the user to check.
- **Returns:** The `address` of the NFT collection contract if a key is found, otherwise returns the `zero address`.

2. Configuration and State View Functions

These functions return the contract’s various configuration structs and system-wide state.

`getStageConstants()`

- **Purpose:** To retrieve the `StageConstants` struct.
- **Returns:** The `StageConstants` struct, containing global values like `maxSellCooldown`, `dailyWindow`, `minBuyAmount`, and `minSellAmount`.

`getFeeConfig()`

- **Purpose:** To retrieve the `FeeConfig` struct.
- **Returns:** The `FeeConfig` struct, containing all fee-related parameters and administrative cooldowns.

getTokenConfig()

- **Purpose:** To retrieve the `TokenConfig` struct.
- **Returns:** The `TokenConfig` struct, containing the addresses of the `baseToken` and `swapToken` and the current `exchangeRate`.

getSystemState()

- **Purpose:** To retrieve the `SystemState` struct.
- **Returns:** The `SystemState` struct, containing data on accumulated fees, admin addresses, and important timestamps.

getStageConfig(UserStage _stage)

- **Purpose:** To retrieve the specific `StageConfig` for a given user stage.
- **Parameter:**
 - `_stage` (`UserStage`): The stage (`PLEB`, `HUSTLER`, or `OG`) to query.
- **Returns:** The `StageConfig` struct with all the parameters for that specific stage.

getExchangeRate()

- **Purpose:** A direct way to get the current token exchange rate.
- **Returns:** The `uint256` value of `tokenConfig.exchangeRate`.

getWhitelistedCollections()

- **Purpose:** To get the list of all whitelisted NFT collection addresses.
- **Returns:** An array of `addresses` representing the NFT contracts that grant access to the vendor.

3. Cooldown Status View Functions

These functions allow anyone to check if administrative cooldowns are currently active.

canChangeFeeRates()

- **Purpose:** To check if the cooldown period for changing fee rates has passed.
- **Returns:** `true` if the `appChangeCooldown` has elapsed since the last fee change, `false` otherwise.

canChangeExchangeRate()

- **Purpose:** To check if the cooldown period for changing the exchange rate has passed.

- **Returns:** `true` if the `rateChangeCooldown` has elapsed since the last rate change, `false` otherwise.
-

DGTokenVendor Smart Contract: Events & Errors

This document provides a reference for all the events and custom errors defined in the `DGTokenVendor` contract. Events are used to log significant actions on the blockchain, while errors explain why a function call failed.

1. Events

Events are signals the contract emits when certain actions occur. Off-chain applications can listen for these events to track activity.

- `TokensPurchased(address indexed buyer, uint256 baseTokenAmount, uint256 swapTokenAmount, uint256 fee)`
 - Emitted when a user successfully buys tokens.
- `TokensSold(address indexed seller, uint256 swapTokenAmount, uint256 baseTokenAmount, uint256 fee)`
 - Emitted when a user successfully sells tokens.
- `Lit(address indexed user, uint256 burnAmount, uint256 newFuel)`
 - Emitted when a user successfully uses the `lightUp` function to gain fuel.
- `StageUpgraded(address indexed user, UserStage newStage)`
 - Emitted when a user successfully upgrades to a new stage.
- `FeesWithdrawn(address indexed to, uint256 baseTokenFees, uint256 swapTokenFees)`
 - Emitted when the developer or owner withdraws accumulated fees.
- `ETHWithdrawn(address indexed to, uint256 amount)`
 - Emitted when the developer or owner withdraws ETH from the contract.

Admin & Configuration Events

- `WhitelistedCollectionAdded(address indexed collectionAddress)`
 - Emitted when a new NFT collection is added to the whitelist during initialization.
- `ExchangeRateUpdated(uint256 newRate)`
 - Emitted when the owner changes the token exchange rate.
- `DevAddressUpdated(address indexed newDevAddress)`
 - Emitted when the developer address is changed.
- `StewardCouncilAddressUpdated(address indexed newStewardCouncilAddress)`
 - Emitted when the owner changes the Steward Council address.

- **FeeRatesUpdated(uint256 newBuyFeeBPS, uint256 newSellFeeBPS)**
– Emitted when the owner updates the buy and sell fee rates.
- **FeeConfigUpdated(uint256 rateChangeCooldown, uint256 appChangeCooldown)**
– Emitted when the owner updates the administrative cooldown periods.
- **StageConfigUpdated(UserStage indexed stage, StageConfig oldConfig, StageConfig newConfig)**
– Emitted when the owner modifies the configuration of a user stage.

2. Custom Errors

Custom errors are used to provide more specific and gas-efficient reasons for a transaction failure compared to simple `require` statements.

User Action Errors

- **MinimumAmountNotMet()**: The amount for a buy or sell is below the required minimum.
- **InsufficientBalance()**: The user does not have enough tokens for the transaction.
- **DailySellLimitExceeded()**: The user has tried to sell more tokens than their daily limit allows.
- **StageSellLimitExceeded()**: The user tried to execute a single sale larger than their stage permits.
- **StageCooldownActive()**: An OG user tried to make a second max-sized sale before the cooldown expired.
- **MaxStageReached()**: A user at the OG stage tried to call `upgradeStage()`.
- **InsufficientPointsForUpgrade()**: The user does not have enough points to upgrade.
- **InsufficientFuelForUpgrade()**: The user does not have enough fuel to upgrade.
- **NoValidKeyForUserFound()**: The caller does not hold an NFT from a whitelisted collection.

Admin Action Errors

- **UnauthorizedCaller()**: The caller is not authorized for a restricted function (e.g., not owner, dev, or admin).
- **AppChangeCooldownStillActive()**: An admin tried to change a setting (like the dev address) before the cooldown expired.
- **FeeCooldownActive()**: An admin tried to change fee rates before the cooldown expired.
- **RateCooldownActive()**: An admin tried to change the exchange rate before the cooldown expired.
- **InvalidFeeBPS()**: The provided fee value is outside the acceptable range (min/max).
- **InvalidDevAddress()**: The new developer address is the zero address.

- `InvalidExchangeRate()`: The new exchange rate is zero or too high.
- `InvalidCooldown()`: The provided cooldown duration is outside the acceptable range.
- `ExceedsMaxWhitelistedCollections()`: Trying to initialize with more NFT collections than the `MAX_WHITELISTED_COLLECTIONS` limit.
- `WhitelistedCollectionsAlreadyInitialized()`: Trying to call `initializeWhitelistedCollections` after it has already been run.

Stage Configuration Errors

- `InvalidFuelRate()`: The `fuelRate` in a new `StageConfig` is invalid.
- `InvalidPointsAwarded()`: The `pointsAwarded` in a new `StageConfig` is invalid.
- `InvalidDailyLimitMultiplier()`: The `dailyLimitMultiplier` in a new `StageConfig` is invalid.
- `InvalidBurnAmount()`: The `burnAmount` in a new `StageConfig` is invalid.
- `InvalidUpgradePointsThreshold()`: The `upgradePointsThreshold` in a new `StageConfig` is invalid.
- `InvalidUpgradeFuelThreshold()`: The `upgradeFuelThreshold` in a new `StageConfig` is invalid.
- `InvalidQualifyingBuyThreshold()`: The `qualifyingBuyThreshold` in a new `StageConfig` is invalid.

System Errors

- `ETHTransferFailed()`: The contract failed to send ETH during a withdrawal.